

End-to-End MLOps Pipeline for Credit Default Risk Prediction with Monitoring, Experiment Tracking, and Operations Console

Abstract

This project presents an end-to-end MLOps solution for predicting loan default risk and operationalizing the machine learning lifecycle. The system encompasses data ingestion, preprocessing, feature engineering, leakage-aware feature selection, model training, evaluation, experiment tracking, model serving, interactive user interface, monitoring, and orchestration.

The implementation integrates reproducible pipelines with deployment-ready services using containerization and observability tools. A unified operations console provides capabilities for inference, system health monitoring, experiment tracking, and pipeline control. The solution demonstrates how a traditional machine learning workflow can be transformed into a scalable, production-oriented system.

3. Problem Statement

Financial institutions require accurate and reliable prediction of credit default risk to minimize financial losses, improve underwriting decisions, and manage portfolio risk. Traditional notebook-based workflows lack reproducibility, scalability, and monitoring capabilities.

This project addresses these limitations by designing a structured MLOps system that enables:

- Automated pipelines
 - Experiment tracking
 - Scalable model serving
 - Real-time monitoring and drift detection
-

4. Use Case

Primary Use Case: Credit Underwriting

- Input borrower financial attributes
- Predict probability of default (PD)
- Classify applicants into risk categories
- Provide explainability for predictions

Secondary Use Cases

- Experiment tracking for model governance
 - System health monitoring
 - Batch prediction for institutional workflows
 - Scheduled pipeline execution
-

5. System Objectives

- Build a reproducible ML pipeline from raw data to deployment
 - Prevent data leakage and ensure valid modeling
 - Provide explainable predictions
 - Track experiments and model versions
 - Deploy using containerized architecture
 - Monitor system performance and data drift
 - Provide a user-friendly UI and operations console
-

6. Project Structure Overview

The repository is modular and organized as follows:

- Data modules (ingestion, preprocessing, feature engineering)
- Model modules (training, evaluation, prediction)
- API layer for serving
- Frontend UI for interaction
- Monitoring stack (Prometheus + Grafana)
- Orchestration (Airflow DAGs)
- Versioning and pipeline management (DVC)

This separation ensures:

- Maintainability

- Scalability
 - Reproducibility
-

7. Dataset Creation and Preparation Methodology

7.1 Raw Data Strategy

The system uses a loan dataset and supports scheduled ingestion. Daily snapshots can be stored to track data evolution and enable reproducibility. If a change in the hash of the data file is detected the data ingestion step is run.

7.2 Target Construction

A binary classification target is derived from loan status:

- Non-default → Class 1
- Default/delinquent → Class 0

Invalid or ambiguous entries are removed to maintain data integrity.

7.3 Cleaning and Transformation

Preprocessing steps include:

- Missing value filtering
 - Date parsing and numeric conversion
 - Encoding categorical variables
 - Feature scaling
-

7.4 Leakage Prevention

Strict leakage prevention is applied:

- Removal of post-loan outcome variables
- Exclusion of time-dependent features that leak future information

This ensures the model reflects real-world deployment conditions.

7.5 Dataset Outputs

The pipeline generates:

- Processed datasets
- Engineered feature matrices
- Train/test splits
- Scaled features
- Selected feature subsets

These artifacts are versioned for consistency and reproducibility.

8. Core Machine Learning Methodology

8.1 Feature Engineering

Feature engineering is a critical step in transforming raw financial and credit data into meaningful representations that improve model performance and generalization. The objective is to construct features that capture borrower behavior, creditworthiness, and risk signals while ensuring robustness and interpretability.

1. Correlation-Based Feature Pruning

Highly correlated features introduce redundancy and can negatively impact model stability, especially for linear models.

- A correlation matrix is computed for all numerical features.
- Pairs of features with correlation above a predefined threshold (e.g., $|\rho| > 0.8$) are identified.
- From each highly correlated pair, one feature is removed based on:
 - Lower predictive importance
 - Higher missingness
 - Lower domain relevance

This reduces multicollinearity, improves model interpretability, and prevents overfitting.

2. Information Value (IV) Analysis and Categorical Utility Checks

Information Value (IV) is used to evaluate the predictive strength of features with respect to the target variable.

- Features are binned (for continuous variables) or grouped (for categorical variables).
- Weight of Evidence (WoE) is computed for each bin.
- IV is calculated as a summary measure of predictive power.

Typical interpretation:

- $IV < 0.02 \rightarrow$ not predictive
- $0.02-0.1 \rightarrow$ weak
- $0.1-0.3 \rightarrow$ medium
- $0.3 \rightarrow$ strong

Features with very low IV are removed, while strong predictors are retained. This ensures that only informative and stable categorical representations are used.

3. Domain-Driven Feature Creation

Domain knowledge from credit risk modeling is incorporated to create meaningful features that reflect borrower behavior.

Examples include:

- Debt-to-Income ratio (DTI) to capture repayment capacity
- Credit utilization ratios (e.g., revolving utilization)
- Account activity metrics (number of open/active accounts)
- Credit history length (derived from earliest credit line)

These features are designed to reflect:

- Borrower repayment ability
 - Credit exposure
 - Historical credit behavior
-

4. Data Transformation and Encoding

To ensure compatibility with machine learning models:

- Categorical variables are encoded using one-hot encoding or binary encoding
 - Percentage values are converted to numeric formats
 - Missing values are handled using mean or median imputation
 - Numerical features are scaled using standardization or normalization
-

5. Leakage-Aware Feature Engineering

Special care is taken to avoid data leakage:

- Post-loan outcome variables are removed
- Time-dependent features that include future information are excluded
- Only features available at loan origination are retained

This ensures that the model reflects real-world deployment conditions.

8.2 Feature Selection

Feature selection aims to identify the most relevant subset of features that contribute significantly to predictive performance while reducing noise and computational complexity.

1. Random Forest-Based Feature Importance

A Random Forest model is used to estimate feature importance scores:

- The model is trained on the processed dataset
 - Feature importance is computed based on:
 - Reduction in impurity (Gini importance)
 - Contribution of each feature across decision trees
-

2. Selection Process

The feature selection process involves:

1. Training a Random Forest model on preprocessed data
2. Ranking features based on importance scores
3. Selecting the top-N features or applying a threshold
4. Removing low-importance features

This results in a compact and high-signal feature set.

3. Benefits of Random Forest-Based Selection

- Captures non-linear relationships
 - Handles feature interactions effectively
 - Robust to outliers and noise
 - Model-agnostic (works well across downstream models)
-

4. Final Feature Set Characteristics

The selected feature set:

- Contains the most predictive variables
 - Reduces dimensionality
 - Improves training efficiency
 - Enhances generalization performance
-

5. Consistency Across Pipeline

To ensure consistency:

- Selected feature names are saved as a manifest
- The same feature order is enforced during inference
- Feature transformations are aligned between training and serving

This guarantees that the model receives identical input structure during deployment.

8.3 Models Trained

- Logistic Regression
- Random Forest
- Gradient Boosting
- Neural Network

This ensures robustness and comparative evaluation.

8.4 Evaluation Methodology

Metrics used:

- Accuracy
- Precision
- Recall
- F1, F2 scores
- Confusion matrix

8.5 Final Model Selected

We selected a Neural Network with hidden layer sizes 128 in layer , 64 in layer 2 trained for 10 epochs with a learning rate of 0.001 using adam optimizer as our final model. The final model is served to the UI and is pointed out using the specific MLFlow model artifact id.

9. Experiment Tracking and Model Lifecycle

Experiment tracking is implemented using MLflow:

- Logs parameters, metrics, and artifacts
- Enables experiment comparison
- Provides reproducibility via run IDs

A metadata file links runtime services to the correct model artifact.

10. Serving Layer and Inference API

The backend is implemented using FastAPI.

Endpoints

- `/health`
- `/predict`
- `/batch_predict`

- [/explain](#)
- [/metrics](#)

Responsibilities

- Load model and features
 - Align input schema
 - Perform preprocessing
 - Return predictions and explanations
-

11. Frontend and Operations Console

Built using Streamlit.

11.1 Prediction Studio

- Single prediction form
 - Batch CSV upload
 - Validation and feedback
 - Probability visualization
 - Explainability outputs
-

11.2 Pipeline Operations Console

- Service health monitoring
 - Pipeline execution triggers
 - Access to MLflow, Grafana, Airflow
 - Operational debugging tools
-

12. MLOps Pipeline and Orchestration

Pipeline Components

- Script execution
- DVC for pipeline DAG
- Apache Airflow for scheduling data ingestion based on changes in the loan_data csv file. It runs the preprocessing creating outputs that can be passed on to the training step.

12.1 DVC Pipeline

- Defines pipeline stages
- Ensures reproducibility
- Supports cached execution
- Tracks data and model versions

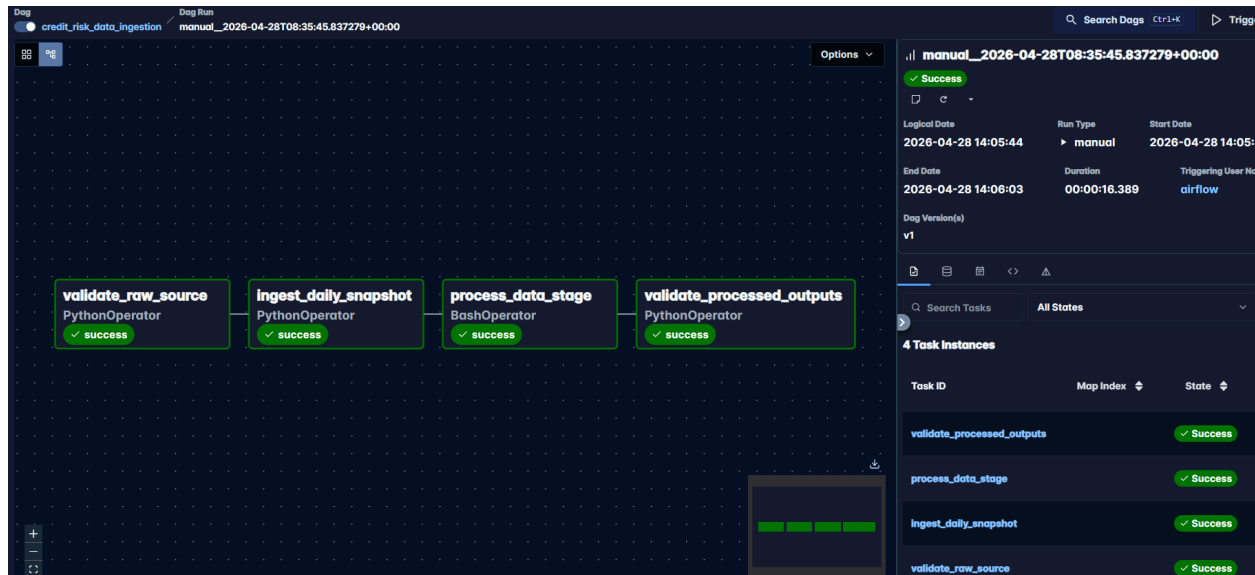
12.2 Airflow Integration

Airflow schedules pipeline execution:

- Runs ingestion pipelines
- Triggers DVC pipeline till the data processing stage
- Logs success/failure states
- Provides pipeline visualization

manual_2026-04-28T08:35:45.837279+00:00		Success	Add a note		Clear Dag Run	Mark Dag Run as...
Logical Date	Run Type	Start Date	End Date	Duration	Triggering User Name	Dag Version(s)
2026-04-28 14:05:44	manual	2026-04-28 14:05:46	2026-04-28 14:06:03	00:00:16.389	airflow	v1

Task Instances							
Task ID	Map Index	State	Start Date	Try Number	Operator	Duration	
validate_processed_outputs		Success	2026-04-28 14:06:02	1	PythonOperator	00:00:00.468	
process_data_stage		Success	2026-04-28 14:05:54	1	BashOperator	00:00:06.982	
ingest_daily_snapshot		Success	2026-04-28 14:05:49	1	PythonOperator	00:00:03.960	
validate_raw_source		Success	2026-04-28 14:05:46	1	PythonOperator	00:00:01.511	



13. Containerized Deployment

Deployed using Docker and Docker Compose.

Services

- Backend API
- Frontend UI
- MLflow
- Prometheus
- Grafana
- Airflow (optional)

Benefits

- Environment parity
 - Reproducibility
 - Scalability
-

14. Monitoring and Observability

14.1 Prometheus Metrics

Tracks:

- Request count
 - Latency
 - Business metrics
 - Drift metrics
-

14.2 Data Drift Detection (PSI-Based)

Data drift is measured using **Population Stability Index (PSI)**:

Population Stability Index (PSI) is a statistical measure used to detect data drift by comparing the distribution of a feature in the training dataset with its distribution in production data. It quantifies how much the input data distribution has shifted over time. A higher PSI value indicates a greater shift and an increased risk of model performance degradation.

For a feature divided into k bins, PSI is calculated as:

$$PSI = \sum (A_i - E_i) \times \ln(A_i / E_i)$$

Where:

- A_i = proportion of observations in bin i for the actual (production) data
- E_i = proportion of observations in bin i for the expected (training) data
- $\ln$ = natural logarithm

Interpretation

- **PSI < 0.1** → No significant drift (stable distribution)
- **0.1 ≤ PSI < 0.25** → Moderate drift (monitor closely)
- **PSI ≥ 0.25** → Significant drift (model may require retraining)

Application in This System

In this project, PSI is used to monitor data drift in input features during model inference. The system compares incoming production data with baseline training distributions and computes

PSI for selected features. These drift scores are logged as monitoring metrics and visualized through dashboards.

When PSI exceeds predefined thresholds, it indicates potential degradation in model performance, and retraining of the model can be triggered as part of the automated pipeline.

14.3 Grafana Dashboards

Visualizes:

- API performance
 - Latency trends
 - Drift metrics
 - Service health
-

15. Testing and Validation

Includes:

- Unit testing
 - API testing
 - Data validation
 - Regression checks
-

16. Pipeline Visualization and Management

The system uses tool-based visualization:

- DVC DAG → pipeline structure
- Airflow UI → execution and scheduling
- MLflow UI → experiment tracking

Airflow serves as a pipeline management console, enabling:

- Execution control
- Error tracking

- Success/failure monitoring
-

Pipeline Performance Metrics

- Execution time per stage
 - Total pipeline latency
 - Throughput (rows processed per second)
-

17. Key Engineering Decisions

- Modular architecture
 - Strict leakage prevention
 - Multi-model benchmarking
 - Separation of frontend and backend
 - Monitoring-first design
 - Config-driven reproducibility
-

18. Challenges and Lessons Learned

- Environment differences in Docker vs local
 - Resource constraints in multi-service systems
 - Service networking complexities
 - Maintaining consistency in explainability
-

19. Conclusion

This project demonstrates a complete MLOps pipeline for credit risk prediction, integrating:

- Reproducible pipelines
- Scalable serving
- Monitoring and drift detection
- Experiment tracking

The system provides a strong foundation for real-world deployment.
